

FULL MARKING SCHEME

IS2200

IS Analysis and Design for Business

Winter 2022 Examination · Paper 1

Module	IS2200 — IS Analysis and Design for Business
Session	Winter 2022
Duration	1.5 hours
Structure	Section A (compulsory) · one question from Section B
Total marks	100 (50 + 50)

SECTION A — COMPULSORY

Question 1

(a) Construct a context level data flow diagram of this system for Sun-Gleam.

(10 marks)

FULL MODEL ANSWER

Scope note. Question 1 is one extended scenario (Sun-Gleam Holidays' booking system) developed across parts (a)–(c). Note this sitting asks only for a context diagram and a Level 1 decomposition (no use case diagram), and devotes a full 10 marks to describing the DFD rules themselves rather than applying them — a different emphasis from other sittings of this paper.

The scenario names five parties that sit outside the automated booking system itself and exchange data across its boundary: the **Customer** (logs in, books, pays), the **Admin Team** (manages bookings, confirms availability, requests payment, receives sales reports), and the three categories of **service provider** named explicitly — **Airlines**, **Hotels**, and **Transport Agencies** — each of which independently confirms availability and receives itinerary-change notifications.

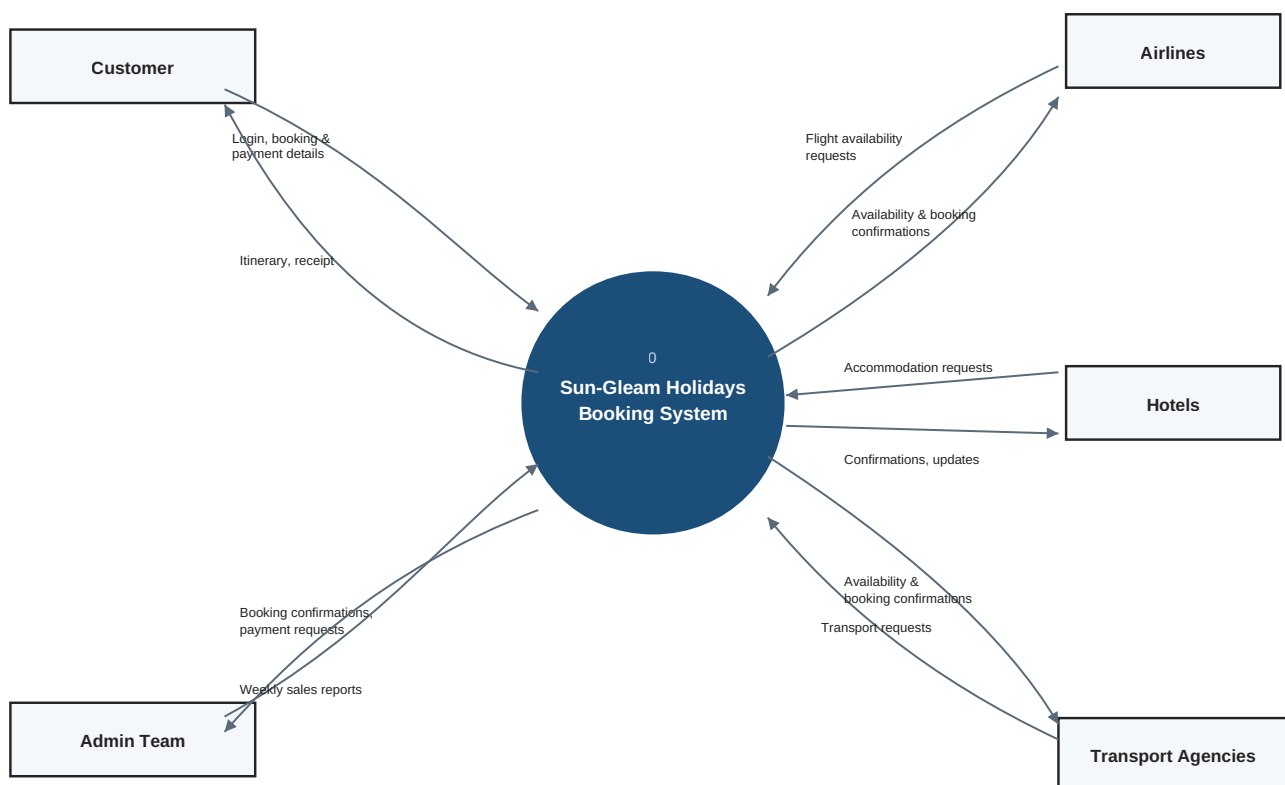


Figure 1(a). Context-level DFD: the Sun-Gleam Holidays booking system as a single process (0), bounded by five external entities.

The three service-provider types are kept as separate entities, rather than merged into one "Service Providers" entity, because the scenario treats them as distinct correspondents each confirming their own portion of a booking (flights, accommodation, and transport are independently available or unavailable) — collapsing them into one entity would

hide that a booking can partially fail (e.g. flights confirmed but transport unavailable), which matters for the exception-handling logic implied by the scenario.

Mark Allocation — (a)

Component	Marks	Criteria for full marks
Single process correctly represents the whole system	2	Exactly one process, numbered 0, labelled with the system name; no internal detail shown.
All five external entities correctly identified	5	1 mark each: Customer, Admin Team, Airlines, Hotels, and Transport Agencies all present and correctly drawn as external entities.
Data flows correctly labelled and directional	2	At least one inbound and one outbound flow per entity, sensibly named from the scenario's own wording.
Correct DFD notation used throughout	1	Circle/rounded shape for the process, squares/rectangles for entities, labelled directional arrows; no data-store symbol at this level.
Total	10	—

(b) Construct a Level 1 data flow diagram of this system for Sun-Gleam.

(30 marks)

FULL MODEL ANSWER

Decomposing process 0 into its major internal processes — while keeping every external entity and flow from part (a) unchanged (the balancing rule) — follows the booking lifecycle described in the scenario from login through to the weekly sales report:

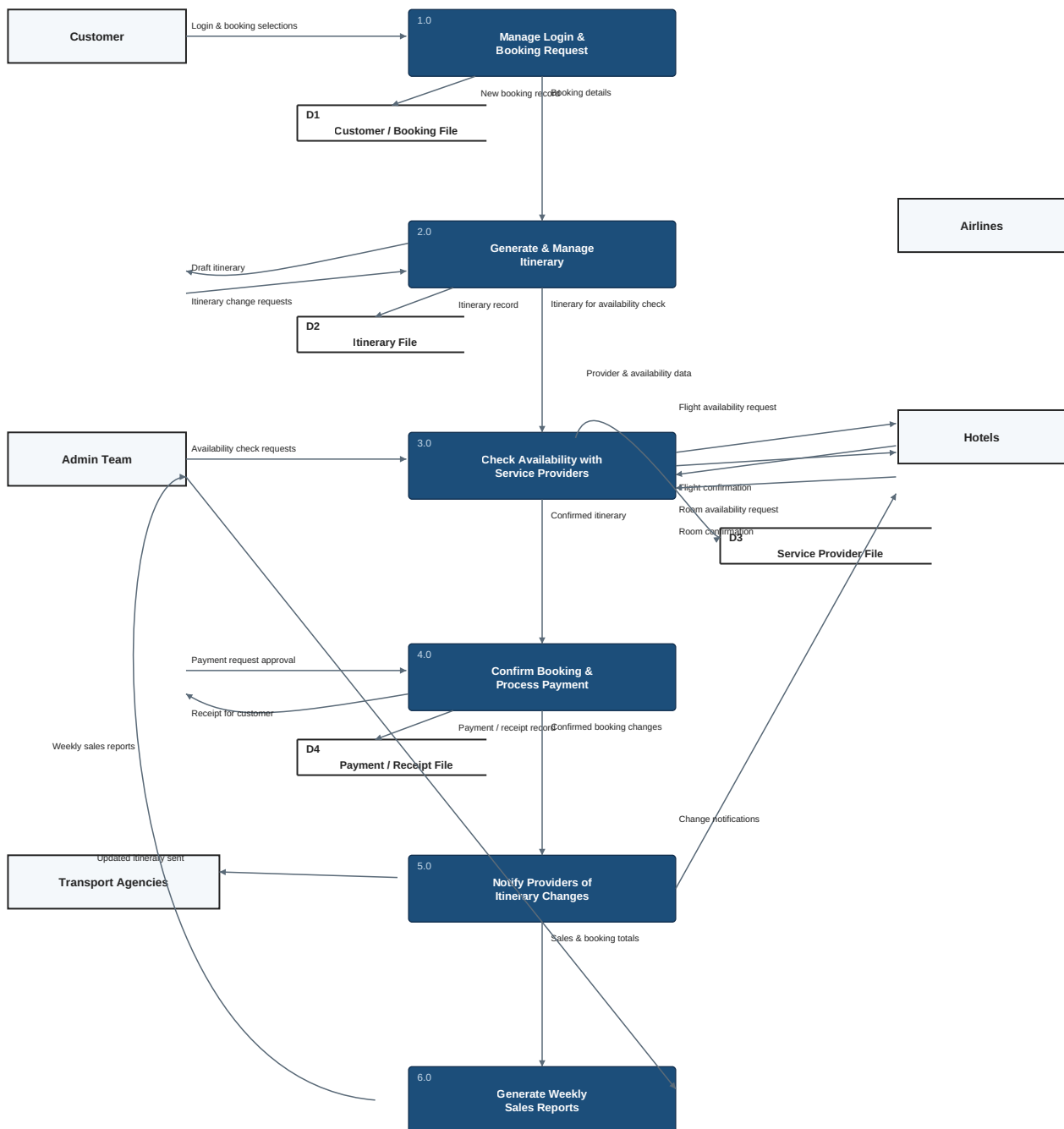


Figure 1(b). Level 1 DFD: decomposition of the Sun-Gleam Holidays booking system into six numbered processes and four data stores.

Process	Covers (from the scenario)
1.0 Manage Login & Booking Request	Customer logs in and selects flights, accommodation, and transport.
2.0 Generate & Manage Itinerary	System generates the itinerary; customer can change booking elements before confirmation.
3.0 Check Availability with Service Providers	Admin team confirms requested flights/accommodation/transport availability with airlines, hotels, and transport agencies.
4.0 Confirm Booking & Process Payment	Admin confirms the itinerary once availability is verified, requests and facilitates payment, generates a receipt.
5.0 Notify Providers of Itinerary Changes	Any customer changes to a confirmed itinerary are sent on to the relevant service providers.
6.0 Generate Weekly Sales Reports	System produces the weekly sales reports the scenario states the admin team should receive.

Four data stores hold the persistent data the scenario implies: **D1 Customer/Booking File** (login and booking-request records), **D2 Itinerary File** (the generated itinerary and any customer changes to it), **D3 Service Provider File** (availability responses and confirmations from airlines, hotels, and transport agencies), and **D4 Payment/Receipt File** (payment requests, facilitation records, and receipts) — with process 6.0 drawing on both D1 and D4 to produce its weekly totals.

Mark Allocation — (b)

Component	Marks	Criteria for full marks
Processes correctly identified, numbered, and decomposed from the context diagram	12	6 major processes present (2 marks each), each a genuine transformation of data, consistently numbered 1.0–6.0, following the scenario's own booking lifecycle.
Data stores correctly identified and labelled	8	D1–D4 all present (2 marks each), each holding data genuinely described as persisted in the scenario, correctly drawn with open-ended store notation.
Data flows accurately connect processes, stores and entities	8	Flows correctly labelled and directed; no data store connected directly to another store or external entity without an intervening process.
Consistency (balancing) with the context diagram in part (a)	1	Same five external entities appear; no external flow from part (a) is dropped or contradicted.
Correct DFD notation throughout	1	Rectangles for processes, open rectangles for stores, squares for entities, labelled directional arrows.
Total	30	—

(c) Describe the rules governing data flow diagramming as discussed in class.

(10 marks)

FULL MODEL ANSWER

Data flow diagramming follows a fixed set of structural rules, grouped by the symbol type they govern, plus a small number of broader guidelines for how diagrams relate to one another across levels:

Symbol / area	Rule
Process	A process cannot have only outputs — it must have at least one input, or there is nothing for it to transform.
	A process cannot have only inputs — it must produce at least one output, or it serves no purpose.
	The inputs to a process must always be different from its outputs — a process must genuinely transform the data, not simply pass it through unchanged.
	Every process has a unique name, so it can always be unambiguously identified and referenced.
Data Store	Data cannot move directly from one data store to another — a process must sit between them.
	Data must always be moved by a process; a data store cannot originate or send data on its own.
	Data cannot move directly from an external entity to a data store without an intervening process.
Source / Sink (external entity)	Data cannot move directly from a source to a sink — it must always pass through a process.
Data Flow	A data flow has only one direction of flow between any two symbols.
	A fork is allowed: the same data can flow from one common location out to two or more different processes, data stores, etc.
	A data flow cannot go back into the same process it leaves — a process cannot feed its own output directly back into itself as an input.

Beyond these symbol-level rules, three broader guidelines govern how diagrams are developed and relate to one another: **completeness** (every data flow must lead somewhere; no store, process, or flow is left dangling or unconnected); **consistency/balancing** (moving from the context diagram to Level 1, and Level 1 to Level 2, must be seamless — every external entity and external data flow shown in a higher-level diagram must still appear, unchanged, in its lower-level decomposition); and **timing** (a DFD is drawn as if the system has never started and will never stop — it shows what the system does, not when or how often, since DFDs are deliberately poor at representing time-based behaviour). A related decomposition guideline is that a single diagram should generally include no more than about nine processes, decomposing further only while doing so still adds genuine clarity rather than confusing complexity.

Mark Allocation — (c)

Component	Marks	Criteria for full marks
Process rules correctly stated	2	At least 3 of the 4 process rules (outputs-only, inputs-only, distinct input/output, unique name) correctly described.
Data store rules correctly stated	2	All 3 data-store rules (no store-to-store, must be moved by a process, no direct entity-to-store flow) correctly described.
Source/sink rule correctly stated	1	Correctly states that data cannot move directly from a source to a sink without passing through a process.
Data flow rules correctly stated	2	All 3 data-flow rules (one direction, forking, no flow back into the same process) correctly described.
Completeness, consistency/balancing, and timing guidelines described	2	All three broader guidelines correctly explained, not just the symbol-level rules.
Decomposition/complexity guideline mentioned	1	Notes the practical limit on process count per diagram and/or the principle of decomposing only while it adds clarity.
Total	10	—

Question 1 Total**50 marks**

SECTION B · OPTION A

SECTION B — ANSWER ONE QUESTION FROM THIS SECTION

Question 2

(a) List the ten steps involved in project planning as described in class, and outline their importance. (20 marks)

FULL MODEL ANSWER

The module's Phase 2 (Project Planning) material sets out ten discrete tasks that together define clear activities and the work needed to complete each one:

#	Step	Why it matters
1	Describe project scope, alternatives, and feasibility	Establishes what the project will and won't cover, checked against operational, technical, schedule, legal, and political feasibility — without this, later estimates have nothing to be scoped against.
2	Divide the project into manageable tasks	A Work Breakdown Structure defines the discrete tasks and their sequence, distinguishing what can run in parallel from what must run sequentially — the foundation every later scheduling step builds on.
3	Estimate resources and create a resource plan	People are the most important and expensive resourcing decision; assigning the right people to the right tasks at the right time materially affects both completion time and system quality.
4	Develop a preliminary schedule	Combines task and resource estimates into target start/end dates via a network diagram, showing task order, dependencies, and what can run in parallel — revisited and refined until acceptable to the client.
5	Develop a communication plan	Project managers spend most of their time communicating; defining who needs what information, when, and in what format up front avoids costly miscommunication later.
6	Determine project standards and procedures	Specifies how deliverables are produced and tested, giving the team a consistent basis to judge itself against and supporting a high-quality outcome and positive user acceptance.
7	Identify and assess risk	Surfaces sources of risk (new technology, user resistance, resource availability, competitive reactions, regulatory change, team experience) early, while there is still time to plan a response.
8	Create a preliminary budget	Turns the resource and schedule estimates into a cost baseline, giving the client and PM a financial yardstick to track spend against as the project proceeds.
9	Develop a scope statement	Formalises the agreed scope, deliverables, and success criteria in writing, so scope creep can be identified and challenged against an explicit, agreed reference point.
10	Set a baseline project plan	Locks in the approved scope, schedule, and budget as the reference point against which all future progress, deviation, and change requests are measured.

Steps 1–7 are covered in detail across this module's Phase 2 (Project Planning) lecture slides, each with its own dedicated content, as reflected in the depth above. Steps 8–10 (preliminary budget, scope statement, baseline project plan) appear in the module's summary list of ten tasks but without further dedicated slides in the material reviewed, so their "why it matters" descriptions above reflect standard project-management practice for these terms rather than a specific class example — check these three against your own lecture notes if your lecturer expanded on them separately.

Mark Allocation — (a)

Component	Marks	Criteria for full marks
Step 1 – Scope, alternatives, feasibility	2	Step correctly named and importance briefly explained.
Step 2 – Divide into manageable tasks	2	Step correctly named and importance briefly explained.
Step 3 – Estimate resources	2	Step correctly named and importance briefly explained.
Step 4 – Preliminary schedule	2	Step correctly named and importance briefly explained.
Step 5 – Communication plan	2	Step correctly named and importance briefly explained.
Step 6 – Standards and procedures	2	Step correctly named and importance briefly explained.
Step 7 – Identify and assess risk	2	Step correctly named and importance briefly explained.
Step 8 – Preliminary budget	2	Step correctly named and importance briefly explained.
Step 9 – Scope statement	2	Step correctly named and importance briefly explained.
Step 10 – Baseline project plan	2	Step correctly named and importance briefly explained.
Total	20	—

(b) Draw a network diagram for the activities.

(20 marks)

FULL MODEL ANSWER

Using the module's own network-diagram convention, each activity is drawn as a labelled node, with a forward pass computing each activity's earliest start/finish (T_E) by summing durations along every predecessor path.

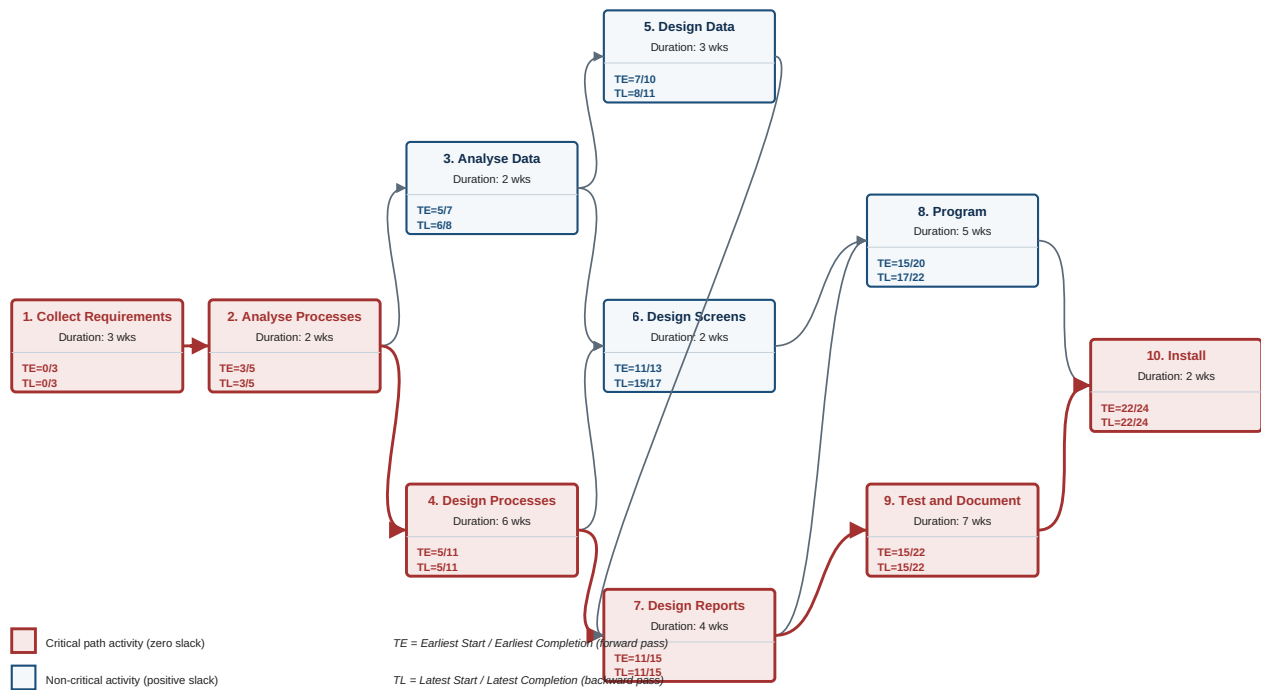


Figure 2(b)/(c). Activity-on-node network diagram for the HR system project, with forward-pass (T_E) and backward-pass (T_L) values shown on every node, and the critical path highlighted in red.

Activity	Description	Duration (wks)	Predecessors	T_E (ES/EF)
1	Collect Requirements	3	—	0 / 3
2	Analyse Processes	2	1	3 / 5
3	Analyse Data	2	2	5 / 7
4	Design Processes	6	2	5 / 11
5	Design Data	3	3	7 / 10
6	Design Screens	2	3, 4	11 / 13
7	Design Reports	4	4, 5	11 / 15
8	Program	5	6, 7	15 / 20
9	Test and Document	7	7	15 / 22
10	Install	2	8, 9	22 / 24

Where an activity has more than one predecessor (e.g. Activity 6 depends on both 3 and 4), its earliest start is the *later* of its predecessors' earliest finish times. The forward pass shows the project's earliest overall completion is **24 weeks** (Activity 10's T_E finish).

Mark Allocation — (b)

Component	Marks	Criteria for full marks
All 10 activities correctly represented with durations	6	Every activity from Table 1 present as a node, correctly labelled with its number, name, and duration.
Precedence relationships correctly drawn	8	Every arrow matches Table 1's predecessor column exactly; no missing or extra dependency; correct handling of multi-predecessor activities (6, 7, 10).
Forward pass (T_B / earliest start & finish) correctly calculated	4	Every node's earliest start/finish correctly computed, taking the maximum of all predecessor finish times where more than one predecessor exists.
Diagram clarity and correct AON notation	2	Node-and-arrow (activity-on-node) format used consistently, matching the module's own network-diagram convention; legible layout with no ambiguous overlaps.
Total	20	—

(c) Show the critical path.

(10 marks)

FULL MODEL ANSWER

The backward pass starts from the project's overall duration (24 weeks, from part (b)) and works right-to-left, computing each activity's latest allowable finish as the *earliest* of its successors' latest start times. Slack (or float) for each activity is latest finish minus earliest finish; any activity with zero slack lies on the critical path, because it has no room to slip without delaying the whole project.

Activity	T _E finish	T _L finish	Slack	On critical path?
1. Collect Requirements	3	3	0	Yes
2. Analyze Processes	5	5	0	Yes
3. Analyze Data	7	8	1	No
4. Design Processes	11	11	0	Yes
5. Design Data	10	11	1	No
6. Design Screens	13	17	4	No
7. Design Reports	15	15	0	Yes
8. Program	20	22	2	No
9. Test and Document	22	22	0	Yes
10. Install	24	24	0	Yes

The critical path is therefore **1 → 2 → 4 → 7 → 9 → 10** (3 + 2 + 6 + 4 + 7 + 2 = **24 weeks**), shown highlighted in red on Figure 2(b)/(c) above. Activities 3, 5, 6, and 8 all carry positive slack and can be delayed by up to that amount without pushing back the project's 24-week completion date.

Mark Allocation — (c)

Component	Marks	Criteria for full marks
Critical path correctly identified	6	States the path 1-2-4-7-9-10 (or equivalent zero-slack justification for each activity on it).
Total project duration correctly stated	2	States 24 weeks, consistent with the forward pass in part (b).
Correct justification via slack/float	2	Explains that critical activities have zero slack ($T_B = T_F$), distinguishing them from the non-critical activities with positive slack.
Total	10	—

Question 2 Total**50 marks**

SECTION B · OPTION B

SECTION B — ANSWER ONE QUESTION FROM THIS SECTION

Question 3

(a) Illustrate both the Cloud Adoption Framework and the Shared Responsibility Model as presented (40 marks) in class and describe the roles of both frameworks in helping organisations with their cloud migration decision.

FULL MODEL ANSWER

Sourcing note. The course pack's own slides on this topic give summary bullet points for both frameworks rather than the full illustrated diagrams themselves (the diagrams are very likely images on the original slides that did not extract as text). The illustrations below reproduce AWS's own published structure for both frameworks — the six-perspective AWS CAF, and the AWS-vs-customer split of the Shared Responsibility Model — cross-checked against AWS's current documentation, and are consistent with the summary bullet points that are present in the course pack (e.g. the Shared Responsibility Model's AWS-side and customer-side items below match the course slide's own list of responsibilities almost exactly).

AWS Cloud Adoption Framework (CAF)

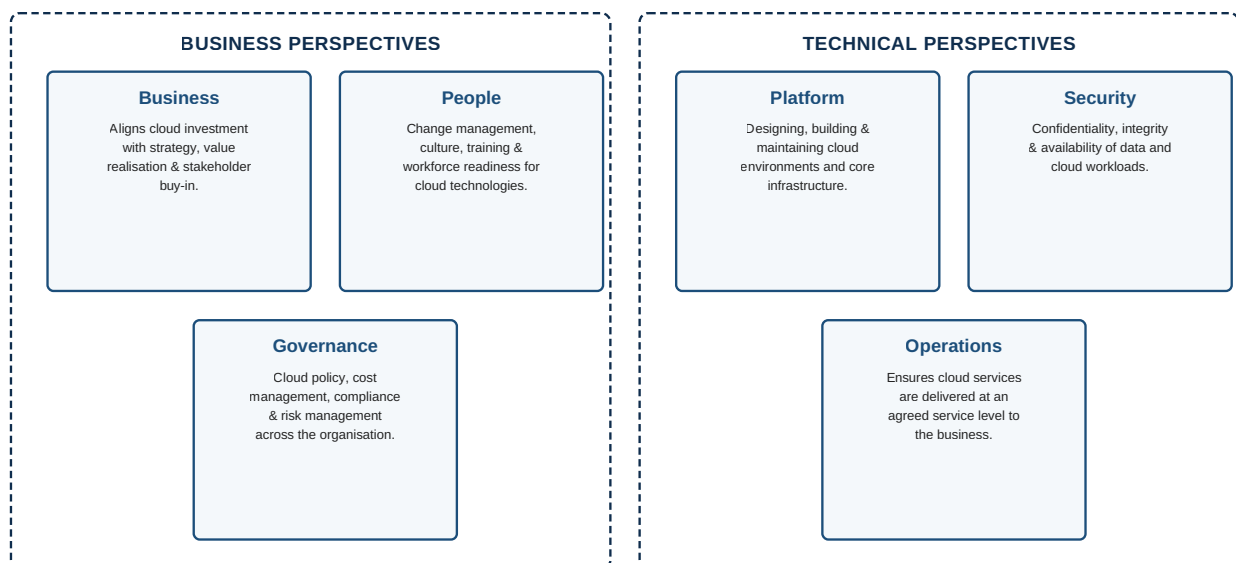


Figure 3(a)-i. AWS Cloud Adoption Framework: six perspectives, grouped into Business and Technical categories.

The **AWS Cloud Adoption Framework (CAF)** is used by thousands of organisations to understand how cloud adoption will transform the way they work. It groups the capabilities an organisation needs into six perspectives — three *business-facing* (Business, People, Governance) and three *technical* (Platform, Security, Operations) — and provides structure to identify and address gaps in an organisation's existing skills and processes. Applying the CAF results in an actionable migration plan with defined work streams, guiding each business unit through updating skills, adapting existing processes, and introducing new ones where needed. In a migration decision specifically, the CAF's role is to answer "are we, as an organisation, ready to adopt the cloud, and where are our gaps?" before a single workload is actually moved.

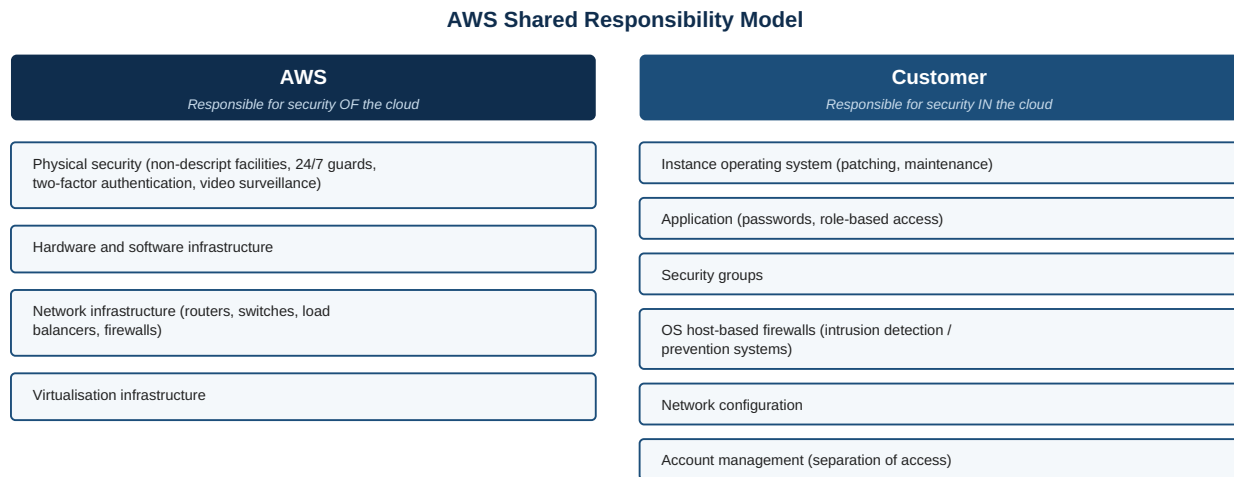


Figure 3(a)-ii. AWS Shared Responsibility Model: security responsibilities split between AWS and the customer.

The **Shared Responsibility Model** addresses a different question — not "are we ready?" but "once we migrate, who is responsible for securing what?" AWS is responsible for the security *of* the cloud: physical data-centre security, the underlying hardware and software infrastructure, network infrastructure, and the virtualisation layer. The customer is responsible for security *in* the cloud: the guest operating system on their instances, their applications, security groups, host-based firewalls, network configuration, and account management. Critically, the specific security steps a customer must take depend on which AWS services they use and how complex their own system is — the more a service is "managed" by AWS, the more of the stack AWS's side of the model covers.

Together, the two frameworks play complementary roles in a migration decision: the CAF assesses *organisational readiness* across business and technical dimensions before migration, while the Shared Responsibility Model clarifies *ongoing accountability* for security once workloads are actually running in AWS — an organisation needs both to make an informed migration decision, since being organisationally ready to migrate (CAF) does not by itself tell a team which security responsibilities they will still own after the move (SRM).

Mark Allocation — (a)

Component	Marks	Criteria for full marks
Cloud Adoption Framework correctly illustrated	10	Six perspectives present and correctly named (Business, People, Governance, Platform, Security, Operations), grouped into business/technical categories.
Cloud Adoption Framework's role correctly described	10	Explains the CAF's purpose (assessing organisational readiness, identifying skill/process gaps, producing an actionable plan) in the context of a migration decision.
Shared Responsibility Model correctly illustrated	10	AWS-side and customer-side responsibilities both correctly listed and clearly separated (at least 3 items per side).
Shared Responsibility Model's role correctly described	10	Explains the security-of-the-cloud vs security-in-the-cloud split, and that responsibility depends on which services/complexity level the customer uses.
Total	40	—

(b) Compare and contrast between AWS data centers, availability zones, and regions.

(10 marks)

FULL MODEL ANSWER

These three terms describe AWS's physical infrastructure at increasing levels of scale, each built to contain and isolate failure from the level above it:

Concept	What it is	Key characteristic
Data Center	A single physical facility, typically housing several thousand servers.	The smallest unit; all data centers are kept online, but in case of failure, automated processes move customer workloads away from the affected facility.
Availability Zone (AZ)	Made up of one or more data centers, interconnected with other AZs using high-speed links.	Designed for fault isolation — no single data center can belong to two AZs, so a failure in one AZ should not cascade into another. AWS recommends replicating across multiple AZs for resiliency, and customers choose which AZ(s) they use.
Region	A cluster of Availability Zones built in a specific geographic area.	As presented in class, AWS's footprint at that time comprised 54 Availability Zones across 18 geographical Regions — the largest-scale grouping, used to keep data close to users or to meet data-residency requirements.

The relationship is therefore strictly nested: a **Region** contains multiple **Availability Zones**, and each **Availability Zone** contains one or more **Data Centers**. Moving up this hierarchy trades a small amount of latency (data centers within the same AZ are closest together) for progressively greater fault isolation — which is why AWS recommends spreading critical workloads across AZs (and, for the highest resilience requirements, across Regions) rather than relying on a single data center or AZ alone.

The specific figures "54 Availability Zones within 18 geographical Regions" reflect AWS's footprint at the time this lecture slide was created. AWS's region/AZ count has grown substantially since (AWS publishes its current, up-to-date figures on its own site), so treat this specific number as the figure "as presented in class" rather than AWS's current footprint.

Mark Allocation — (b)

Component	Marks	Criteria for full marks
Data Center correctly defined	3	Correctly describes it as the smallest physical unit (a single facility housing many servers).
Availability Zone correctly defined	4	Correctly describes an AZ as one or more data centers, designed for fault isolation, interconnected via high-speed links.
Region correctly defined and relationship to AZs/data centers clear	3	Correctly describes a Region as a geographic cluster of AZs, and correctly states the nested Region > AZ > Data Center relationship.
Total	10	—

Question 3 Total

50 marks